

Reviewer Pack — Free Entropy Gap in Disjointness Communication Matrices

Entry ba26a4ca7784 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 07:51:08 UTC

Free Entropy Gap in Disjointness Communication Matrices

Entry ID: ba26a4ca7784 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 07:51:08 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Communication Complexity of Disjointness

Statement:

For a random Disjointness matrix $M \in \{0,1\}^{\{n \times n\}}$, let $\chi(M)$ denote the free entropy of its entries treated as non-commutative random variables. Then $\chi(M) \geq \Omega(\sqrt{n})$ holds with probability ≥ 0.95 over random seeds, and $\chi(M) = \Theta(\sqrt{n})$ implies the randomized communication complexity of M is $\Omega(n)$.

Rationale (proposer's reasoning):

Free entropy quantifies the 'non-commutative complexity' of matrix entries, capturing dependencies that correlate with communication bottlenecks. By framing Disjointness matrices as non-commutative distributions, we expose structural constraints that may lower-bound communication requirements via free probability's asymptotic freeness properties.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bb3ab84d08d23e03

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 16
    M = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    def free_entropy(M):
        trace = sum(M[i][i] for i in range(n))
        return -trace / n

    chi_M = free_entropy(M)

    metric_name = "free_entropy"
    metric_value = chi_M
    instances_tested = 1
    conjecture_holds = chi_M >= 4 * math.sqrt(n)
    counterexample = "" if conjecture_holds else f"chi(M) < 4√n"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
```

```

support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not res["conjecture_holds"] for res in results):
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='chi(M) < 4\\n' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

lue': -0.5625, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'chi(M) < 4\\n'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': -0.625, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'chi(M) < 4\\n'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': -0.625, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'chi(M) < 4\\n'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': -0.4375, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'chi(M) < 4\\n'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': -0.4375, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'chi(M) < 4\\n'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': -0.5, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'chi(M) < 4\\n'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': -0.5625, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'chi(M) < 4\\n'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': -0.5625, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'chi(M) < 4\\n'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': -0.375, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'chi(M) < 4\\n'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': -0.5, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'chi(M) < 4\\n'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a045ef7b.py", line 60, in <module>
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
                        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a045ef7b.py", line 60, in <genexpr>
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
                        ~~~~~^~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before completing validation; counterexamples show failure but cannot trust results due to crash | next: Fix seed handling in test code and re-run with sufficient sample size

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136470
2	propose	ollama_remote	qwen3:8b	0	0	146803
3	propose	ollama_remote	qwen3:8b	0	0	138280
4	propose	ollama_remote	qwen3:8b	0	0	48484
5	preregistration	ollama_remote	qwen3:8b	0	0	31711
6	novelty	ollama_remote	qwen3:8b	0	0	27283
7	novelty	ollama_remote	qwen3:8b	0	0	19098
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12642
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10470
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6005
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6224
12	judge	ollama_remote	qwen3:8b	0	0	25007

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 608476 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/ba26a4ca7784.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ba26a4ca7784.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ba26a4ca7784.tar.gz` (if generated)